

UNIVERSITÉ LAVAL

Faculté de sciences et génies

Planification de Maintenance Ferroviaire : Comparaison de Deux Modèles MiniZinc sur Chuffed et Gecode

IFT-4001 IFT-7020

Présenté à
Claude-Guy Quimper

par

Ethan Facca, Léa Potin

Matricule	**Nom**
537 403 134	Ethan Facca
537 396 798	Léa Potin

Table 1 – Matricules et noms des membres de l'équipe

14 décembre 2025

1 Introduction

La planification de maintenance ferroviaire représente un défi opérationnel majeur pour les entreprises de transport, notamment la SNCF. Avec des centaines de trains nécessitant des maintenances régulières, des fenêtres temporelles strictes, et des ressources limitées (voies, techniciens, équipements), ce problème présente toutes les caractéristiques d'un problème d'optimisation combinatoire complexe.

Ce projet s'intéresse à la modélisation de ce problème comme un **Resource-Constrained Project Scheduling Problem (RCPSP)** avec des contraintes spécifiques au domaine ferroviaire : fenêtres temporelles strictes (possiblement multiples par train), temps de setup variables selon le contexte, et ressources multiples avec capacités simultanées.

1.1 Problématique

Le problème consiste à planifier les opérations de maintenance sur un horizon de plusieurs jours en respectant :

— Les fenêtres temporelles de présence des trains au centre de maintenance
— Les contraintes de précédence entre opérations avec temps de setup
— Les capacités limitées des ressources (voies exclusives, techniciens par type, équipements)
— Les contraintes de travail des techniciens (temps min/max, pauses)
— Les compatibilités entre voies et équipements
— La gestion dynamique des deadlines et des maintenances optionnelles

L'objectif est de minimiser le retard pondéré des maintenances par rapport à leurs dates butoirs, calculées en fonction de leur périodicité et de la dernière fois où elles ont été effectuées.

1.2 Objectifs

Les objectifs de ce projet sont :

1. Modéliser le problème de planification de maintenance ferroviaire comme un RCPSP avec contraintes temporelles
2. Développer deux modèles MiniZinc distincts : l-v1 (Léa) avec vérification ponctuelle des ressources, et e-v1 (Ethan) avec modélisation par blocs de travail
3. Comparer les performances des deux modèles sur trois solveurs (Chuffed, Gecode, OR-Tools) sur différentes instances
4. Identifier les limites de résolvabilité selon la complexité des instances
5. Analyser l'impact des choix de modélisation (vérification ponctuelle vs blocs de travail) sur la performance

1.3 Structure du rapport

Ce rapport est organisé comme suit : la Section 2 présente la description formelle du problème, la Section 3 décrit les deux modèles développés et leurs différences, la Section 4 présente le protocole expérimental, la Section 5 expose les résultats expérimentaux, la Section 6 en discute les implications, et la Section 7 conclut le travail.

2 Description du problème

Le problème de planification de maintenance ferroviaire consiste à organiser les opérations de maintenance sur un horizon temporel donné, tout en respectant de nombreuses contraintes de ressources et de disponibilité des trains. L'objectif principal est de minimiser les retards des maintenances par rapport à leurs dates butoirs, en tenant compte des priorités.

2.1 Entités et paramètres

Les **trains** constituent l'élément central : chaque train possède une ou plusieurs fenêtres de présence au centre de maintenance, définies par des temps d'arrivée et de départ. Les trains sont caractérisés par leur type, qui détermine leurs propriétés techniques (étage requis, complexité influençant les temps de setup).

Les **maintenances** sont organisées en types, chacun défini par :

— Une **périodicité** : intervalle entre deux exécutions (0 pour urgente, ∞ pour non applicable)
— Un **ensemble d'opérations** requises
— Une **priorité/pondération** pour la fonction objectif

Chaque **opération** spécifie :

— Une **durée** dur_o (variable selon le type de train)
— Des **relations de précédence** avec d'autres opérations
— Des **exigences en ressources** : type de technicien, équipements requis, compatibilité avec certains types de voies

Les **ressources** se répartissent en trois catégories :

— **Voies** : caractérisées par leur type et étage, ne peuvent accueillir qu'une opération à la fois. Certains équipements ne sont disponibles que sur certaines voies.
— **Techniciens** : organisés par types (spécialités) avec un nombre limité N_k par type et des contraintes de temps de travail ($T_{min,k}$, $T_{max,k}$ par jour).
— **Équipements** : quantité totale et capacité simultanée maximale C_e (nombre d'opérations pouvant l'utiliser simultanément).

Les **paramètres temporels** incluent :

— L'horizon de planification (plusieurs jours)
— L'unité temporelle (minutes ou périodes)
— Deux temps de setup : σ_{op} (entre opérations d'un même train) et σ_{train} (pour les changements de trains, plus long)

2.2 Variables de décision

Pour résoudre le problème, on doit déterminer :

— s_o, f_o : temps de début et de fin de chaque opération o
— s_m, f_m : temps de début et de fin de chaque maintenance m
— x_o : voie assignée à chaque opération o

Selon la variante du problème, d'autres variables peuvent être nécessaires : allocation spécifique de techniciens aux opérations, activation/désactivation de maintenances optionnelles, horaires de travail détaillés des techniciens.

2.3 Contraintes

Une solution valide doit respecter les contraintes suivantes :

Contraintes temporelles :

— Cohérence : $f_o = s_o + dur_o$ pour chaque opération o

— Fenêtres : les maintenances dans les fenêtres de présence des trains

— Précédence : $s_o \geq f_{pred(o)} + \sigma_{op}$ si l'opération o a un prédécesseur $pred(o)$

— Hiérarchie : opérations dans la fenêtre de leur maintenance

Contraintes de ressources : À tout instant, le nombre de techniciens de type k utilisés $\leq N_k$ et l'utilisation simultanée d'équipement $e \leq C_e$. Les techniciens respectent les temps min/max par jour et ne peuvent effectuer qu'une tâche à la fois (ubiquité).

Contraintes de compatibilité : Équipements disponibles sur les voies choisies, types de techniciens correspondant aux besoins des opérations.

Contraintes de non-chevauchement : Deux opérations o_1 et o_2 sur la même voie (i.e., $x_{o_1} = x_{o_2}$) doivent être disjointes :

$$f_{o_1} + \sigma \leq s_{o_2} \text{ ou } f_{o_2} + \sigma \leq s_{o_1} \tag{1}$$

où $\sigma = \sigma_{op}$ si même train, $\sigma = \sigma_{train}$ sinon.

2.4 Objectif

L'objectif est de minimiser le retard pondéré :

$$\min \sum_m w_m \cdot \max(0, f_m - \delta_m) \tag{2}$$

où w_m est un poids reflétant la priorité de la maintenance m, et δ_m sa deadline (calculée comme la dernière fois où elle a été effectuée plus sa périodicité). Les maintenances non planifiées peuvent recevoir une forte pénalité selon la variante du problème.

3 Approche proposée

Deux modèles MiniZinc ont été développés : **l-v1** (Léa) avec vérification ponctuelle des ressources, et **e-v1** (Ethan) avec modélisation fine par blocs de travail. Les deux utilisent les solveurs standards (Chuffed, Gecode, OR-Tools) sans heuristiques personnalisées.

3.1 Modèle l-v1 : Vérification ponctuelle

Architecture **monolithique** (1 fichier) utilisant les **minutes** comme unité temporelle. Chaque train a une **fenêtre unique** définie par son arrivée et son départ.

Gestion des ressources par vérification ponctuelle : Au lieu de créer une variable pour chaque technicien à chaque instant (ce qui nécessiterait un nombre de variables proportionnel au produit du nombre d'opérations, de types de techniciens, et de l'horizon temporel), le modèle vérifie simplement qu'à chaque début d'opération, la demande totale en techniciens ne dépasse pas la capacité disponible. Concrètement, pour chaque type de technicien et chaque opération, on compte combien de techniciens de ce type sont requis par toutes les opérations en cours à cet instant, et on

vérifie que ce total ne dépasse pas le nombre disponible. Cette approche réduit le nombre de variables à seulement les temps et voies des opérations. La même logique s'applique aux équipements.

Simplifications : Toutes les maintenances doivent être planifiées (pas d'optionnalité), les pauses des techniciens ne sont pas modélisées, et les contraintes de temps minimum de travail sont désactivées pour éviter l'infaisabilité. L'objectif minimise simplement le retard pondéré par priorité.

3.2 Modèle e-v1 : Modélisation fine et modulaire

Architecture **modulaire** (6 fichiers) utilisant les **périodes** comme unité temporelle (typiquement 15 minutes). L'horizon total P (en périodes) est le produit du nombre de jours J et du nombre de périodes par jour PJ.

Intervalles multiples : Contrairement à l-v1 où chaque train a une seule fenêtre de présence, un train peut avoir plusieurs intervalles de présence durant l'horizon. Par exemple, un train peut arriver le matin, repartir l'après-midi, puis revenir le soir. Le modèle gère automatiquement les temps passés en gare qui sont retranchés de la fenêtre utilisable pour la maintenance.

Instances dynamiques de maintenances : Le modèle génère automatiquement des instances de maintenances avec possibilité de ne pas les planifier. Pour chaque instance i, une variable booléenne varMActive[i] détermine si elle est effectuée. Les maintenances urgentes (périodicité nulle) sont obligatoires, les autres optionnelles. Les deadlines (dates butoirs) se propagent dynamiquement : si une maintenance est effectuée, la prochaine deadline est calculée à partir de son temps de fin plus la périodicité ; sinon, elle utilise la valeur historique. Cela permet une gestion réaliste où reporter une maintenance décale automatiquement la suivante.

Variables d'affectation : En plus des variables de temps et voies, le modèle définit pour chaque instance de maintenance i et opération o une variable varOAffectation[i, o] indiquant quel technicien spécifique (parmi tous les techniciens disponibles) est assigné à cette opération. La valeur 0 signifie que l'opération n'est pas applicable.

Blocs de travail des techniciens : Ce qui distingue principalement e-v1 de l-v1. Le modèle planifie explicitement les horaires de chaque technicien : chaque jour est divisé en 2 à 4 blocs de travail continus séparés par des pauses. Les contraintes imposent que les blocs d'un même jour ne se chevauchent pas, que la somme des durées respecte les temps min/max quotidiens, et que chaque opération soit entièrement contenue dans un bloc (pas de tâche à cheval sur une pause). Cette modélisation permet de gérer de manière réaliste les contraintes de travail avec pauses obligatoires, mais augmente considérablement la complexité du modèle.

Objectif avec pénalités : Contrairement à l-v1 qui considère toutes les maintenances obligatoires, e-v1 gère les maintenances optionnelles. Pour chaque instance active, on calcule le retard pondéré habituel. Pour chaque instance non planifiée, on applique une forte pénalité (10000) multipliée par le poids de la maintenance. Les maintenances urgentes ont un poids de 100, les préventives un poids de 1. Cela encourage fortement à planifier toutes les maintenances tout en permettant d'en omettre si vraiment nécessaire.

Aspect	l-v1	e-v1
Organisation	Monolithique	Modulaire
Unité temporelle	Minutes	Périodes
Fenêtres/train	1 fixe	Multiples
Maintenances	Obligatoires	Optionnelles
Deadlines	Statiques	Dynamiques
Gestion techniciens	Vérif. ponctuelle	Blocs détaillés
Pauses	Non	Oui (1-3)
Affectation individuelle	Non	Oui
Complexité	Faible	Élevée

TABLE 2 – Comparaison l-v1 vs e-v1

3.3 Comparaison

4 Protocole d'expérimentation

Cette section décrit le protocole expérimental pour comparer les deux modèles développés sur trois solveurs MiniZinc (Chuffed, Gecode et OR-Tools).

4.1 Instances du banc d'essai

Huit instances de test ont été générées automatiquement à l'aide d'un script Python paramétrable, couvrant différentes tailles et complexités :

- **Small :** 2 instances
  - `small_01` : 2 trains, 2 types d'opérations, 2 types de maintenances
  - `small_02` : 3 trains, 2 types d'opérations, 2 types de maintenances
- **Medium :** 3 instances
  - `medium_01` : 5 trains, 3 types d'opérations, 2 types de maintenances
  - `medium_02` : 7 trains, 3 types d'opérations, 2 types de maintenances
  - `medium_03` : 5 trains, 3 types d'opérations, 3 types de maintenances
- **Large :** 3 instances
  - `large_01` : 10 trains, 3 types d'opérations, 2 types de maintenances
  - `large_02` : 10 trains, 3 types d'opérations, 3 types de maintenances
  - `large_03` : 15 trains, 3 types d'opérations, 2 types de maintenances

Toutes les instances partagent les mêmes paramètres temporels :

- Horizon de planification : 1 jour (144 périodes de 10 minutes = 1440 minutes)
- Temps de setup entre opérations : 1 période (10 minutes)
- Temps de sécurité entre trains sur une voie : 4 périodes (40 minutes)

Chaque instance comprend également des ressources variables (voies de types différents avec étages caténaires, équipements avec capacités limitées, techniciens de différentes spécialités avec contraintes de temps de travail) ainsi qu'un mélange de maintenances urgentes et préventives avec des périodicités et échéances différentes.

4.2 Métriques

Pour chaque instance et chaque solveur, nous mesurons :

— **Temps de résolution :** Temps total pour trouver une solution optimale ou atteindre le timeout (en secondes)
— **Statut :** Optimal, satisfiable, insatisfiable (UNSAT), ou timeout
— **Qualité de la solution :** Valeur de la fonction objectif (minimisation du retard pondéré) pour les instances résolues

4.3 Environnement

Les tests sont effectués sur une machine virtuelle Windows 11 (Oracle VirtualBox) hébergée sur un système avec processeur Intel Core i7-1260P et 16 Go de RAM. La machine virtuelle est configurée avec 4 cœurs CPU et 6 Go de RAM allouée.

Les solveurs utilisés sont :
— **Chuffed 0.13.2** : solveur lazy clause generation
— **Gecode 6.3.0** : solveur à base de propagation de contraintes avec 4 threads
— **OR-Tools CP-SAT 9.14.6206** : solveur SAT avec 4 threads

Le temps limite est fixé à 600 secondes (10 minutes) par instance. Pour chaque combinaison instance-modèle-solveur (8 instances $\times$ 2 modèles $\times$ 3 solveurs = 48 combinaisons), les métriques sont collectées et enregistrées.

5 Résultats

Cette section présente les résultats expérimentaux de la comparaison des deux modèles sur trois solveurs : Chuffed, Gecode et OR-Tools, avec un timeout de 10 minutes (600 secondes).

5.1 Performance globale

Le Tableau 3 présente les temps de résolution pour chaque combinaison instance-modèle-solveur. Les timeouts (dépassement du temps limite de 600 secondes) sont indiqués par **TO**, et les résultats non encore disponibles par —.

Table 3 – Temps de résolution (secondes) pour chaque combinaison instance-modèle-solveur

	l-v1 (Léa)			**e-v1 (Ethan)**		
Instance	Chuffed	Gecode	OR-Tools	Chuffed	Gecode	OR-Tools
small_01	5.1	0.4	0.4	1.8	0.8	1.7
small_02	5.1	0.4	0.5	8.8	**TO**	4.4
medium_01	4.7	0.6	0.9	37.1	—	22.1
medium_02	0.5	0.5	0.6	5.0	—	6.7
medium_03	94.0	**TO**	5.5	**TO**	—	42.3
large_01	0.9	0.9	1.5	8.2	—	16.9
large_02	14.7	**TO**	45.4	**TO**	—	**TO**
large_03	238.0	**TO**	5.9	**TO**	—	175.0

Note : TO = Timeout ($>$ 600s), — = Résultats non encore disponibles. Les temps sont arrondis à la première décimale.

5.2 Qualité des solutions

Le Tableau 4 présente les valeurs de la fonction objectif (retard pondéré total) obtenues pour chaque instance résolue.

TABLE 4 – Valeurs de la fonction objectif pour chaque combinaison instance-modèle-solveur

	l-v1 (Léa)			**e-v1 (Ethan)**		
Instance	Chuffed	Gecode	OR-Tools	Chuffed	Gecode	OR-Tools
small_01	2300	2300	2300	72090	72090	72090
small_02	2300	2520	2520	5722	—	5722
medium_01	1720	1720	1720	23938	229864	23938
medium_02	0	0	0	0	—	0
medium_03	6010	—	6010	—	—	120019
large_01	0	0	0	0	—	0
large_02	0	—	0	—	—	—
large_03	10360	—	10360	—	—	23791

Important : Ces valeurs ne sont **pas directement comparables** entre les deux modèles pour plusieurs raisons :

- Les **instances diffèrent** : e-v1 et l-v1 utilisent des fichiers .dzn distincts générés à partir des mêmes configurations, mais avec des formats de données différents adaptés à chaque modèle.
- Les **contraintes diffèrent** : e-v1 inclut des contraintes strictes sur les blocs de travail, pauses, et temps min/max des techniciens que l-v1 n'a pas.
- Les **fonctions objectif diffèrent** : l-v1 minimise le retard simple pondéré par priorité, tandis que e-v1 utilise des poids différenciés (100 pour urgente, 1 pour préventive) et applique des pénalités (10000) pour les maintenances non planifiées.
- Les **deadlines diffèrent** : e-v1 calcule dynamiquement les deadlines avec propagation, tandis que l-v1 utilise des deadlines statiques.

Néanmoins, ces valeurs permettent de vérifier que les solveurs trouvent des solutions cohérentes pour un même modèle (par exemple, tous les solveurs trouvent 0 pour large_01 avec l-v1).

5.3 Comparaison des modèles

Modèle l-v1 (Léa) : Le modèle l-v1 montre des performances globalement meilleures sur les instances small et medium. Sur les 8 instances testées, il résout toutes les instances small en moins de 6 secondes avec tous les solveurs. Les instances medium sont également bien gérées, avec des temps variant de 0.5s à 94s selon le solveur. Les instances large présentent plus de difficultés, avec 2 timeouts sur Gecode (large_02, large_03).

Modèle e-v1 (Ethan) : Le modèle e-v1 présente des temps de résolution systématiquement plus élevés que l-v1, reflétant sa complexité accrue due à la modélisation fine des blocs de travail des techniciens. Sur les instances small, les temps varient de 0.8s à 8.8s, soit environ 2 à 20 fois plus lents que l-v1. Les instances medium et large présentent 4 timeouts (small_02 avec Gecode, medium_03 avec Chuffed, large_02 et large_03 avec Chuffed).

Impact de la modélisation : La différence de performance s'explique par le nombre de variables : l-v1 utilise $O(|OP|)$ variables (temps et voies uniquement), tandis que e-v1 ajoute $O(|S| \times J \times B)$ variables pour les blocs de travail des techniciens, où S est le nombre de techniciens, J le nombre de

jours, et B le nombre de blocs par jour. Cette explosion combinatoire rend la résolution significativement plus difficile, au prix d'une modélisation plus réaliste des contraintes humaines.

5.4 Comparaison des solveurs

Pour l-v1 :

— **OR-Tools** : Le plus performant globalement, aucun timeout. Excellentes performances sur large_03 (5.9s) et medium_03 (5.5s) où les autres échouent ou sont lents.

— **Gecode** : Très rapide sur small (<0.5s) et certaines medium (0.5-0.6s), mais 3 timeouts sur les instances plus complexes.

— **Chuffed** : Performances intermédiaires, mais parfois lent sur small (5.1s) comparé aux autres. Timeout sur medium_03 (94s) mais résout large_03 en 238s.

Pour e-v1 :

— **OR-Tools** : Le plus robuste avec seulement 1 timeout (large_02). Résout toutes les instances small et medium, ainsi que large_01 et large_03.

— **Chuffed** : 3 timeouts (medium_03, large_02, large_03). Temps élevés même sur medium (37s pour medium_01).

— **Gecode** : Sur small, timeout sur small_02 (>600s) mais résout small_01 en 0.8s.

Observation générale : OR-Tools se révèle le solveur le plus stable sur les deux modèles, avec une capacité à résoudre des instances que les autres abandonnent. Gecode excelle sur les petites instances de l-v1 mais peine sur e-v1 et les grandes instances. Chuffed montre des performances variables selon l'instance.

6 Discussion

Cette section discute les résultats expérimentaux et présente les découvertes concernant le comportement des trois solveurs (Chuffed, Gecode et OR-Tools) sur les deux modèles.

6.1 Performance des solveurs

Les résultats montrent des différences importantes entre les trois solveurs, avec des comportements qui varient selon le modèle utilisé.

OR-Tools : Le solveur le plus robuste OR-Tools se distingue comme le solveur le plus performant et stable sur les deux modèles. Pour l-v1, il résout toutes les instances sans timeout, avec des temps particulièrement remarquables sur les instances difficiles (large_03 en 5.9s, medium_03 en 5.5s). Pour e-v1, il résout 7 instances sur 8, avec seulement 1 timeout sur large_02. Cette robustesse suggère qu'OR-Tools gère mieux la complexité combinatoire des deux modèles, probablement grâce à ses techniques avancées de propagation et de recherche.

Gecode : Performances variables selon le modèle Gecode montre un comportement très différent selon le modèle. Pour l-v1, il excelle sur les petites instances (small_01 et small_02 en 0.4s) et certaines instances medium (medium_01 et medium_02 en 0.5-0.6s), mais échoue avec 3 timeouts sur les instances plus complexes (medium_03, large_02, large_03). Pour e-v1, les résultats sont limités : il résout uniquement small_01 en 0.8s et échoue sur small_02 avec un timeout. Cette

différence suggère que Gecode est sensible à la complexité du modèle e-v1, probablement due à l'explosion du nombre de variables liées aux blocs de travail des techniciens.

Chuffed : Performances intermédiaires et imprévisibles Chuffed présente des performances variables et parfois surprenantes. Pour l-v1, il est relativement lent sur les petites instances (5.1s pour small_01 et small_02) comparé aux autres solveurs, mais résout large_03 en 238s alors que Gecode échoue. Pour e-v1, il résout 4 instances sur 8, avec 3 timeouts (medium_03, large_02, large_03). Les temps sont également élevés même sur des instances résolues (37.1s pour medium_01). Cette variabilité suggère que Chuffed peut être efficace sur certaines structures de problèmes mais moins prévisible que OR-Tools.

6.2 Comparaison des modèles

L'analyse comparative révèle un impact significatif de la modélisation sur les performances, avec des compromis clairs entre réalisme et résolubilité.

Performance globale : l-v1 plus rapide Le modèle l-v1 montre des performances globalement supérieures à e-v1. Sur les instances small, l-v1 résout toutes les instances en moins de 6 secondes avec tous les solveurs, tandis qu'e-v1 nécessite jusqu'à 8.8s et présente 1 timeout. Sur les instances medium et large, l-v1 résout 6 instances sur 8 sans timeout, tandis qu'e-v1 présente 4 timeouts. Cette différence s'explique par la complexité combinatoire : l-v1 utilise $O(|OP|)$ variables (temps et voies), tandis qu'e-v1 ajoute $O(|S| \times J \times B)$ variables pour les blocs de travail des techniciens.

Instances où un modèle excelle Il existe des cas où un modèle excelle et l'autre échoue. Par exemple, e-v1 résout large_01 en 8.2s avec Chuffed et 16.9s avec OR-Tools, tandis que l-v1 le résout en 0.9s avec Chuffed et Gecode, et 1.5s avec OR-Tools. Inversement, sur medium_03, l-v1 résout en 5.5s avec OR-Tools, tandis qu'e-v1 présente un timeout avec Chuffed mais résout en 42.3s avec OR-Tools. Ces différences suggèrent que la structure des instances peut favoriser un modèle ou l'autre selon les contraintes dominantes.

Impact de la modélisation sur la performance La modélisation fine des techniciens dans e-v1 (blocs de travail, pauses, temps min/max) augmente significativement la complexité du problème. Cette complexité se traduit par des temps de résolution 2 à 20 fois plus élevés sur les instances small, et par 4 timeouts sur les instances medium et large. Cependant, cette modélisation offre une représentation plus réaliste des contraintes humaines, ce qui peut être crucial pour des applications pratiques où la conformité aux règles de travail est essentielle.

6.3 Facteurs de difficulté

L'analyse des résultats révèle plusieurs facteurs qui influencent la difficulté de résolution :

Taille des instances Les instances small (8-12 opérations) sont toutes résolubles avec tous les solveurs, avec des temps variant de 0.4s à 8.8s selon le modèle et le solveur. Les instances medium (30-45 opérations) présentent des temps plus variables (0.5s à 94s) et quelques timeouts, notamment avec Gecode sur l-v1 et Chuffed sur e-v1. Les instances large (60-90 opérations) sont les plus difficiles, avec plusieurs timeouts sur les deux modèles, particulièrement avec Gecode et Chuffed.

Impact du solveur Le choix du solveur a un impact majeur sur la résolubilité. OR-Tools résout 15 instances sur 16 (94%) sans timeout, tandis que Gecode résout 11 instances sur 16 (69%) et

Chuffed résout 12 instances sur 16 (75%). Cette différence suggère que les techniques de propagation et de recherche d'OR-Tools sont mieux adaptées à ce type de problème.

Impact de la modélisation Le modèle e-v1 présente systématiquement plus de difficultés que l-v1, avec 4 timeouts contre 2 pour l-v1. Cette différence s'explique par l'explosion combinatoire due aux variables de blocs de travail des techniciens. Cependant, les deux modèles montrent des patterns similaires : les instances small sont faciles, les medium sont modérées, et les large sont difficiles.

Structure des contraintes Certaines instances présentent des valeurs d'objectif de 0 (large_01 et large_02 avec l-v1, medium_02 et large_01 avec e-v1), indiquant qu'il est possible de planifier toutes les maintenances sans retard. D'autres instances présentent des retards significatifs (jusqu'à 23791 pour large_03 avec e-v1), suggérant que les contraintes sont serrées mais satisfaisables.

6.4 Limites et perspectives

Limites de l'étude Cette étude présente plusieurs limites. Premièrement, le nombre d'instances est limité (8 instances), ce qui peut limiter la généralisation des conclusions. Deuxièmement, certains résultats pour e-v1 avec Gecode ne sont pas encore disponibles (indiqués par — dans les tableaux), ce qui empêche une comparaison complète. Troisièmement, les instances utilisent des formats de données différents pour chaque modèle, ce qui rend difficile une comparaison directe des valeurs d'objectif. Quatrièmement, le timeout de 600 secondes peut être trop court pour certaines instances difficiles qui pourraient être résolues avec plus de temps.

Perspectives futures Plusieurs directions de recherche futures sont prometteuses. Premièrement, générer plus d'instances avec différentes caractéristiques (ratio ressources/demande, densité des contraintes, distribution des priorités) permettrait de mieux comprendre les facteurs de difficulté. Deuxièmement, développer des heuristiques personnalisées pour chaque modèle pourrait améliorer les performances. Enfin, adapter les instances pour qu'elles soient identiques entre les deux modèles permettrait une comparaison plus directe des valeurs d'objectif.

7 Conclusion

Ce projet a permis de développer et comparer deux modèles distincts pour le problème de planification de maintenance ferroviaire, évalués sur trois solveurs MiniZinc (Chuffed, Gecode et OR-Tools).

Les contributions principales sont : (1) le développement de deux modèles MiniZinc avec des approches différentes (modélisation fine des techniciens vs vérification ponctuelle), (2) l'évaluation sur 8 instances de différentes tailles, (3) la comparaison des performances sur trois solveurs, et (4) l'identification des facteurs de difficulté.

Les résultats révèlent que OR-Tools est le solveur le plus robuste (15 instances sur 16 résolues), que le modèle l-v1 montre des performances supérieures à e-v1 (temps 2 à 20 fois plus rapides), mais que la complexité combinatoire de e-v1 impacte significativement les performances (4 timeouts vs 2). Toutes les instances small sont résolubles, tandis que les instances medium et large présentent des difficultés variables selon le modèle et le solveur.

Les perspectives futures incluent générer plus d'instances, développer des heuristiques personnalisées, et adapter les instances pour permettre une comparaison directe entre modèles.

A Annexe : Code et ressources

A.1 Modèles MiniZinc

Les codes complets des deux modèles MiniZinc sont disponibles dans le dossier `models/` :
Modèle l-v1 (Léa) : Architecture monolithique disponible dans `models/l-v1/model.mzn`
Modèle e-v1 (Ethan) : Architecture modulaire disponible dans `models/e-v1/` avec :

— `main.mzn` : point d'entrée principal
— `generic.mzn` : énumérations et constantes génériques
— `constant.mzn` : paramètres d'entrée
— `computed.mzn` : calculs préliminaires
— `constraints/maintenances.mzn` : contraintes sur les maintenances
— `constraints/techniciens.mzn` : contraintes sur les techniciens et blocs de travail

A.2 Générateur d'instances

Les outils de génération d'instances sont disponibles dans le dossier `instances/generator/` :

— `generer_data.py` : génère les fichiers de données (.dzn) pour les deux modèles à partir de configurations
— `creer_config.py` : crée les fichiers de configuration des instances

A.3 Instances de test

Les instances de données sont disponibles dans le dossier `instances/` avec 8 instances de tailles variées :

— **Small** : small_01, small_02 (2 trains, 2-3 maintenances)
— **Medium** : medium_01, medium_02, medium_03 (3-4 trains, 4-5 maintenances)
— **Large** : large_01, large_02, large_03 (5-6 trains, 6-8 maintenances)

Chaque instance existe en deux versions : `*_l-v1.dzn` pour le modèle l-v1 et `*_e-v1.dzn` pour le modèle e-v1. Le dossier `instances/configs/` contient les configurations JSON utilisées pour générer ces instances.

Instances multi-jours (e-v1 uniquement) : Des instances supplémentaires exploitant les capacités spécifiques du modèle e-v1 (intervalles multiples, horizons étendus sur plusieurs jours) sont disponibles dans le dossier `instances/multiday/` avec leurs configurations dans `instances/configs/multiday/`.

A.4 Résultats expérimentaux

Les résultats des expérimentations sont organisés dans le dossier `results/` par modèle et par solveur :

— `results/l-v1/chuffed/`, `results/l-v1/gecode/`, `results/l-v1/or-tools/`
— `results/e-v1/chuffed/`, `results/e-v1/gecode/`, `results/e-v1/or-tools/`

Chaque fichier de résultat contient les sorties complètes de MiniZinc incluant les temps de résolution, les solutions trouvées, et les statistiques d'exécution.

Résultats multi-jours (e-v1 uniquement) : Les résultats pour les instances multi-jours démontrant les capacités avancées du modèle e-v1 sont disponibles dans un dossier séparé :

— `results/e-v1/multiday/chuffed/`
— `results/e-v1/multiday/gecode/`
— `results/e-v1/multiday/or-tools/`